arXiv:2607.17269v1 [cs.AI] 19 Jul 2026

An Explicit World Model Based on Data-First Ontology: DaoQL Multimodal Storage Validation and Counterfactual Reasoning Evaluation

Zhanbo Li, Shifeng Wu, Xiangjin Meng, Wenjie Cai
Graduate Students, School of Mathematics and Statistics, Chongqing University
DaoQL Project Group
Contact affiliation: Guangzhou Polytechnic Normal University
shifengwu@gpnu.edu.cn
zhanbo.lee@hotmail.com
202506131076T@stu.cqu.edu.cn
202506131075T@stu.cqu.edu.cn

May 2026

Abstract

Large language models encode world models implicitly in neural weights, which exposes four structural risks in high-precision domains such as medicine and finance: hallucination, frozen knowledge, poor explainability, and poor modifiability. This paper proposes *data-first ontology*: LLMs are treated as reasoning and language engines, while deterministic knowledge is moved into an explicit multimodal database, DaoQL. We formalize an explicit world model and show that, under rule independence, deterministic evaluation, and fixed conflict resolution, explicit models provide a sufficient condition for composable counterfactual decomposability; implicit models lack atomic read/δ semantics and therefore provide no comparable architectural guarantee. The implemented system focuses on DaoQL's verified storage layer and explicit Eval path, integrating graph, column, vector, and full-text engines within one process. KVCache graph nodes, expert hot updates, and the DaoQL-Agent runtime remain future work. On an embedded same-machine setup, DaoQL reports graph BFS at 1.20 ms, HNSW at 83.1 μs, and a Fluent hybrid query at 105.8 μs; these results indicate engineering potential but must be interpreted with deployment-shape differences from client-server systems. Exploratory measurements on LDBC SNB SF1 [15] and ANN-Benchmarks [2] further show 34/34 query coverage with interactive-class queries mostly in the sub-millisecond to millisecond range, but only 1.8 QPS overall due to long-tail BI/IC queries; ANN-Benchmarks reaches Recall@10 $\geq$ 99% at thousand-level QPS after a bridge-edge protection fix. In a five-domain counterfactual experiment ($n = 1250$), DaoQL+GPT-4o achieves 94% composable counterfactual decomposability, 49 percentage points above GPT-4o alone. The paper explicitly separates provable structure, preliminary empirical evidence, and architectural roadmap claims.

Keywords: data-first ontology; multimodal database; neuro-symbolic AI; world model; coun-

terfactual reasoning; benchmark evaluation

1 Introduction

Large language models (LLMs) [12, 9] show strong generalization and natural-language reasoning, but their default paradigm encodes a "world model" implicitly in billions of neural parameters [8, 13]. This tight coupling of knowledge and computation creates four structural limitations in settings that require precision and auditability: factual conflicts caused by probabilistic sampling, frozen knowledge with high update cost, weak explainability of reasoning paths [3], and poor surgical modifiability due to catastrophic interference [17]. We argue at the operational-semantics level that these limitations share one gap: implicit models lack atomic read-by-ID and local update semantics.

RAG and agent frameworks such as LangChain, Hermes, and OpenClaw mitigate memory and knowledge defects by attaching vector stores or graphs to LLMs. This establishes a separation between the logic layer and the storage layer, but it still has three bottlenecks. First, token windows force lossy context compression. Second, memory retrieval and agent execution often live in separate processes, introducing synchronization and serialization overhead. Third, reasoning logs and database writes are usually recorded separately, making financial-grade auditability difficult. Existing systems do not provide read, δ, Eval, and Trace as unified atomic semantics.

We propose an explicit world-model architecture guided by *data-first ontology* [5, 25, 27, 1]: the LLM is a reasoning and natural-language engine, while deterministic knowledge is stored in DaoQL, a multimodal database. DaoQL uses six explicit primitives - Being, Def, Type, Relation, Contract, and Version - to construct an addressable and versioned world model W. Theoretically, we formalize the operational-semantics difference between explicit and implicit models, derive a sufficient condition for composable counterfactual decomposability (Theorem 1), and analyze why single-process multimodal execution can reduce latency. Empirically, the paper focuses on the verified storage layer, explicit Eval path, and counterfactual experiment; KVCache graph nodes and the full DaoQL-Agent runtime remain design-stage components.

The contributions are:

1. **Theory.** We define an explicit world model, prove finite Def bootstrapping, strong Contract termination, and local update, and explain why implicit models lack an architectural guarantee for composable counterfactuals.

2. **System.** We implement and evaluate DaoQL's multimodal storage layer, including mmap reads, relation-type CSR adjacency, WAL/MVCC, and cross-engine hybrid queries.

3. **Evaluation.** We combine three-tier microbenchmarks, same-machine competitor comparisons, LDBC/ANN exploratory standard-set measurements, and a 1,250-query counterfactual experiment.

Table 1: Scope of claims in this paper

Layer / capability	Evidence in this paper	Status
Explicit world-model theory	Formal definitions, well-formedness, counterfactual sufficient conditions	Established theoretically
DaoQL storage layer	Graph, column, vector, full-text, WAL/MVCC, mmap benchmarks	Implemented and measured
LDBC SNB / ANN sets	SF1 34/34 query coverage; ANN Recall@10 curve	Exploratory measurement
Explicit Eval path	1,250-query counterfactual experiment	Preliminary validation
Memory-layer recall	HNSW + Tantivy + graph benchmarks as indirect support	Partially supported
DaoQL-Agent runtime	Design document and context mechanism	Design stage
KVCache graph nodes and expert hot updates	Architecture roadmap	Future work

2 Related Work

2.1 Knowledge Representation and Graph Databases

Knowledge representation and reasoning (KRR) [16, 20, 14, 18, 24, 23, 6] aims to make entities and logic machine-operable. RDF [32] and OWL [30, 31] provide the logical foundation of the Semantic Web, but mainly describe static propositions and do not natively integrate version chains or executable behavioral semantics. Datomic [26] pioneered immutable append and "time as a first-class concern," but its EAVT model remains a passive fact-recording model rather than an autonomous computational unit model.

Property graphs such as Neo4j [19, 29] and strongly typed graph databases such as TypeDB [28] improve structured graph modeling. However, for next-generation AI world models they still face two limitations: inheritance and schema are largely static, and vector retrieval and column aggregation are not unified in the same process, causing network and serialization overhead. DaoQL addresses this design space by embedding typing, graph traversal, vector search, column aggregation, versioning, and contract checks into one storage kernel.

2.2 LLM-Augmentation and Agent Frameworks

RAG systems increasingly attach external knowledge bases to LLMs. GraphRAG [4] and LightRAG [7] extract entities and relations from text and use graph topology to improve context. Yet the graph remains an external, mostly read-only augmentation component. When injected facts conflict with parameterized knowledge, systems still lack deterministic, atomic conflict resolution and counterfactual evaluation.

Long-context memory systems such as MemGPT [21] manage conversation history using a

virtual-memory analogy, but operate at the text-block granularity rather than as structured entities with version chains. Agent frameworks typically use LLMs as controllers and external databases such as SQLite, Chroma, or Pinecone as memory components. This is easy to engineer but fragments context, memory, and audit logs across systems. DaoQL-Agent proposes representing turns and messages as Being nodes and using graph traversal instead of lossy compaction; however, the full agent runtime is not measured in this paper.

2.3 Industrial Ontology Platforms and Standard Benchmarks

Industrial Ontology platforms, exemplified by Palantir Foundry [22], use Object/Link/Action semantics to model enterprise entities and operations. They share the paper's motivation toward explicit semantics, but typically place Ontology as a semantic mapping layer above multiple stores such as Cassandra, Elasticsearch, and JanusGraph. Their advantage is federated access to existing data infrastructure. DaoQL instead sinks Being/Type/Relation into single-process storage primitives, trading migration cost for a tighter Eval/Trace path. LDBC SNB [15] and ANN-Benchmarks [2] provide common workloads for graph and approximate nearest-neighbor systems; we use them as exploratory standard-set measurements rather than official certification.

3 Materials and Methods

This section is organized as theory, implementation, and experimental design. Section 3.1 formalizes the explicit world model; Section 3.2 describes the system architecture; Section 3.3 specifies benchmarks and counterfactual evaluation.

3.1 Theory and Formal Model

3.1.1 Overview of Data-First Ontology

Data-first ontology claims that a world model should consist of structured, inspectable, controllable explicit data rather than implicit weights. DaoQL realizes this through six primitives. **Being** is the basic entity unit and is identified by UUID. **Def** and **Type** define fields, constraints, relationships, and runtime dispatch. **Relation** connects Beings as directed edges. **Contract** is a fuel-limited Rhai script embedded in the write path. **Version** is a hash-linked history record that supports MVCC and audit replay. The following formal framework gives the precise definitions and proof obligations.

To turn the Introduction's "knowledge sinking" claim into testable scientific statements, this section formalizes the explicit world model and derives its structural properties; system implementation (§3.2) and experimental design (§3.3) build on this foundation.

3.1.2 Formal Definitions

An implicit world model encodes knowledge in a neural network $f_\theta : \mathcal{X} \to \mathcal{Y}$ [8, 13], with forward inference and global parameter updates $\theta \mapsto \theta'$ as primary operations. An explicit world model, by contrast, carries deterministic knowledge in addressable, versioned data structures. Table 2 summarizes the main symbols.

Table 2: Main symbols in the theory framework

Symbol	Meaning
$\mathcal{ID}, \mathcal{B}, \mathcal{D}, \mathcal{T}, \mathcal{R}, \mathcal{C}, \mathcal{V}$	Identifier and six primitive sets
$W = (\mathcal{B}, \mathcal{D}, \mathcal{T}, \mathcal{R}, \mathcal{C}, \mathcal{V})$	Explicit world-model state
$\delta(W, p)$	State after applying patch p to W
$\mathsf{Eval}(\phi, W)$	Evaluation of query/rule ϕ on state W
$\mathsf{Trace}(q, W)$	Execution trace of query q on W
$F_{\max}$	Contract fuel limit

Definition 1 (Six Primitives)**.** *The primitives are:*

- ***Being*** $b = (id, core, ext, history)$*: $id \in \mathcal{ID}$, $history = [v_1, \ldots, v_n]$ is a version chain;*
- ***Def*** $d = (name, fields, constraints, contracts, fsm, rel_decls)$*;*
- ***Type*** $t = (name, parent, contracts, constraints)$*;*
- ***Relation*** $r = (type, src, dst, direction, ext, window)$*;*
- ***Contract*** $c = (hook, source, ast)$*;*
- ***Version*** $v = (ts, hash, prev_hash, payload)$*.*

The explicit world model is $W = (\mathcal{B}, \mathcal{D}, \mathcal{T}, \mathcal{R}, \mathcal{C}, \mathcal{V})$.

Definition 2 (State Equivalence)**.** *Given a fact domain $\mathcal{F}$, if W_1 and W_2 agree on all observable facts in $\mathcal{F}$, we write $W_1 \equiv_{\mathcal{F}} W_2$.*

This structure realizes four atomic semantics: read by ID, patch update (δ), deterministic rule evaluation (Eval), and execution tracing (Trace). DaoQL implements them via BeingId addressing, append-only WAL, and MVCC snapshots (§3.2.3).

3.1.3 Well-Formedness and Execution Safety

DaoQL adopts a "schema-as-data" design: Def is itself a Being, bootstrapped by the axiom Def `DAO_DEF`. We must ensure that this mechanism does not yield infinite meta-definition chains and that Contracts embedded in the write path cannot violate system termination.

Proposition 1 (Finite Def Bootstrap Chain)**.** *Let axiom Def $d_0 \in \mathcal{D}$ satisfy $d_0.core.def = d_0$; and $\forall d \in \mathcal{D} \setminus \{d_0\}$, $d.core.def = d_0$, $\forall b \in \mathcal{B}$, $b.core.def \in \mathcal{D} \setminus \{d_0\}$. Then any ordinary Being has def-reference chain length at most 2, any ordinary Def has chain length 1, and `DAO_DEF` is only a controlled self-reference.*

Proof. For any $b \in \mathcal{B}$, $b.core.def = d_1 \in \mathcal{D} \setminus \{d_0\}$ and $d_1.core.def = d_0$, hence $b \to d_1 \to d_0$ with chain length 2. For any $d \in \mathcal{D} \setminus \{d_0\}$, $d \to d_0$ with chain length 1. Suppose an infinite chain $d_0 \to d_1 \to d_2 \to \cdots$ exists: if $d_1 \neq d_0$, then d_1 is an ordinary Def and $d_1.core.def = d_0$ forces termination at d_0, a contradiction. □

Proposition 2 (Strong Contract Termination)**.** *Suppose Contract execution forbids new writes and external I/O, consumes one fuel unit per step, and is capped at* $F_{\max}$*. Then* $\forall c \in \mathcal{C}$*,* $\mathsf{Exec}(c)$ *halts within* $F_{\max}$ *steps.*

Proof. Use remaining fuel $\Phi(e_t) = \mathsf{Fuel}(e_t)$ as a potential function, initialized to $F_{\max}$ and strictly decreasing by one per step. Forbidding write recursion and external I/O excludes infinite-wait paths; by well-ordering of natural numbers, execution terminates and rolls back when fuel reaches zero, so at most $F_{\max}$ steps occur. □

Propositions 1–2 bound the metadata layer and guarantee write-path termination, supporting the `DAO_DEF` axiom layer and the default 10,000-step Contract fuel limit described in §3.1.1.

3.1.4 Operational Semantics: Implicit vs. Explicit Models

The four LLM crises named in the Introduction can be unified at the operational-semantics level: implicit model $\mathcal{O}_{\text{imp}} = \{f_\theta,\, \theta \mapsto \theta'\}$ lacks database-style atomic read and δ.

Observation 1 (Implicit Models Lack Database-Style Atomic Operations)**.** *Operationally, LLM knowledge is encoded in distributed parameters* θ*, with usable operations mainly* $f_\theta(x)$ *and* $\theta \mapsto \theta'$*; therefore existing LLM architectures do not provide database-isomorphic* $\mathsf{read}_\theta(id)$ *and* $\delta_\theta(\theta, patch_{id})$*, nor can precise read-by-ID and local update be system invariants.*

Proof. If a δ_θ existed that updated only the fact for id, parameters should decompose as $\theta = \bigsqcup_{id} \theta|_{id}$; but LLM weights encode semantics in a distributed manner, with no provably independent $\theta|_{id}$. Fine-tuning $\theta' = \theta - \eta \nabla_\theta L$ is global and aligns with catastrophic interference [17]. Prompt intervention $y = f_\theta(x')$ changes input only, not persistent state. The assumption fails. □

Proposition 3 (Local Update in Explicit Models)**.** *Let* $W' = \delta(W, p)$*, where the scope of* p *is* $\mathsf{Dom}(p) \subseteq \{id_0\} \cup \{r \mid r.src = id_0 \text{ or } r.dst = id_0\}$*, and the system uses BeingId addressing, append-only WAL, and MVCC snapshot reads. Then* $\forall b \in \mathcal{B}$*,* $b.id \neq id_0 : W'|_b \equiv W|_b$*.*

Proof. The patch does not touch other entity fields; WAL appends only version v_{n+1} for id_0, leaving $history(b)$, $core(b)$, and $ext(b)$ unchanged for all other entities. By the definition of state equivalence, semantics on the untouched fact domain is unchanged. □

Observation 2 (Shared Operational Root of the Four LLM Crises)**.** *If a system has only* $\mathcal{O}_{\text{imp}}$ *without* read*,* δ*,* Eval*, and* Trace*, then hallucination resistance (needs deterministic* Eval*), live updates (needs* δ*), explainability (needs* Trace*), and surgical edits (needs local* δ*) cannot simultaneously hold as architectural invariants.*

Proof. By Observation 1, implicit models lack local δ and atomic read. f_θ outputs token distributions rather than $\mathsf{Eval}(\phi, S) \in \{\top, \bot\}$ [11]; $\theta \mapsto \theta'$ is costly and global; internal activations cannot be reconstructed as symbolic reasoning chains. The operations required by all four properties are absent from $\mathcal{O}_{\text{imp}}$, hence they share one operational root. □

Observation 2 does not claim that every LLM failure has a single explanation; rather, in settings requiring deterministic factual judgment, local update, and audit trails, the four crises

share one operational gap. Data-first ontology fills these atomic semantics through explicit structure W, forming the theoretical starting point of our architecture.

3.1.5 Counterfactual Reasoning

In Pearl's causal hierarchy, the counterfactual layer asks how conclusions would change had facts differed [23]. We operationalize this as applying patch p to explicit state $S = (W, \mathcal{K})$ and executing $\mathsf{Eval}(\phi, \delta(S, p))$, which directly matches the counterfactual protocol in §3.3.

Theorem 1 (Composable Counterfactual Decomposability)**.** *Let p_1, p_2 be patches composed of finitely many atomic update/relate operations, and let ϕ depend on fact domain $\mathcal{F}_\phi$. If $\delta(\delta(S, p_1), p_2) \equiv_{\mathcal{F}_\phi} \delta(\delta(S, p_2), p_1)$, Eval is deterministic, and rule conflicts have fixed priority, then*

$$\mathsf{Eval}(\phi, \delta(\delta(S, p_1), p_2)) = \mathsf{Eval}(\phi, \delta(S, p_1 \oplus p_2)). \tag{1}$$

Proof. Let $S_2 = \delta(\delta(S, p_1), p_2)$ and $S_{12} = \delta(S, p_1 \oplus p_2)$. By patch commutativity and fixed conflict resolution, $S_2 \equiv_{\mathcal{F}_\phi} S_{12}$. Since ϕ depends only on $\mathcal{F}_\phi$, evaluation is a deterministic function $g_\phi(S|_{\mathcal{F}_\phi})$, hence $\mathsf{Eval}(\phi, S_2) = \mathsf{Eval}(\phi, S_{12})$. □

Observation 3 (No Architectural Guarantee of Composable Counterfactuals for Implicit Models)**.** *LLM implicit evaluation $\hat{\phi}(\ell, S) = Sample(f_\theta(\ell \oplus encode(S)))$ cannot architecturally guarantee that semantically equivalent phrasings yield the same conclusion, that composed edits satisfy Theorem 1, or that edit order is immaterial.*

Proof. On the explicit path, semantically equivalent phrasings ℓ_1, ℓ_2 map to the same patch $p = \pi(\ell_1) = \pi(\ell_2)$; on the implicit path, $\ell_1 \neq \ell_2$ may yield $P_\theta(y|\ell_1, S) \neq P_\theta(y|\ell_2, S)$. Implicit models lack δ, so comparable structured states S_2 and S_{12} cannot be constructed; prompt order changes attention context, so commutativity has no architectural guarantee. □

Theorem 1 shows that under rule independence and deterministic Eval, composable counterfactual decomposability is a provable structural property; Observation 3 explains why the pure LLM path lacks an equivalent guarantee. Together they set the theoretical expectations for §4.1.

3.1.6 Neuro-Symbolic Division of Labor and Performance Mechanism

In a neuro-symbolic architecture, deterministic knowledge $G \subseteq \mathcal{S}$ lives on the symbolic side; queries are evaluated by $\mathsf{Eval}(\phi, \mathcal{S})$ and the LLM only naturalizes $y = \pi_{nl}(r)$. The executor records $\mathsf{Trace} = \langle op_1, \ldots, op_m \rangle$ with Being/Relation IDs accessed at each step, so conclusions trace back to explicit reasoning paths—explaining 100% traceability in the counterfactual experiment. This property fails when the LLM bypasses Eval and generates directly.

For performance, hybrid query $q = q_{\text{vec}} \circ q_{\text{graph}} \circ q_{\text{col}}$ in a multi-stack architecture requires k cross-process calls. With per-call network/IPC latency $T_{\text{net},i} \geq \tau_{\text{net}} > 0$,

$$T_{\text{multi}} - T_{\text{mono}} \geq k\tau_{\text{net}} - T_{\text{dispatch}}, \tag{2}$$

where $T_{\text{multi}} = \sum_i (T_{\text{net},i} + T_{\text{ser},i} + T_{\text{query},i})$ and $T_{\text{mono}} = T_{\text{vec}} + T_{\text{graph}} + T_{\text{col}} + T_{\text{dispatch}}$. In DaoQL's single process $T_{\text{net}} = 0$, eliminating cross-system overhead on the order of $k\tau_{\text{net}}$; this inequality

explains low hybrid-query latency but does not imply DaoQL is asymptotically optimal on all algorithms.

For graph traversal, DaoQL pointer-adjacency BFS on subgraph $G' = (V', E')$ runs in $O(|V'| + |E'|)$ because each vertex and edge is processed at most once. This shows the storage shape approximates the ideal BFS access model; measured 4.2–6.7× speedup over Neo4j (Table 8) is an engineering constant factor and must not be conflated with the complexity result.

We distinguish two kinds of claims: Contract termination, local update, and Eq. (1) are provable under stated axioms; BFS 1.20 ms and 94% composable decomposability are empirical results on specific hardware and datasets, validating predictions rather than replacing theory.

3.2 System Architecture and Implementation

3.2.1 Five-Layer Integrated Architecture

Figure 1 illustrates how an explicit world model could grow into a full neuro-symbolic system. The empirically verified core in this paper is the storage layer and the explicit Eval path; several memory, learning, inference, and application components remain roadmap items.

- **Storage layer.** DaoQL integrates graph, column, vector, and full-text engines in one process with a unified BeingId. WAL and MVCC provide atomicity and crash consistency.
- **Memory layer.** Session lifecycle and historical recall are intended to use HNSW, Tantivy, and graph traversal. These components are individually measured, but the full agent memory layer is not.
- **Learning layer.** Expert and router hot updates are planned architecture goals.
- **Inference layer.** A customized LLM can in principle navigate semantically related KV-Cache graph nodes, but this remains a planned direction.
- **Application layer.** DaoQL-Agent exposes FSM-driven workflows and tool-safety Contracts, but the runtime is design-stage.

In the complete design, a user query triggers hybrid recall, locates relevant Beings and KVCache offsets, loads them through mmap, invokes inference, and writes results atomically through the WAL. The verified part is concentrated in storage: the current configuration bypasses PageCache/ArcSwap on graph reads, and WAL recovery for 10K entries takes 219.6 μs.

3.2.2 Graph-Driven Agent Context and Direct mmap Reads

External agent frameworks usually compress old messages because of token-window limits. DaoQL-Agent instead models interaction history as graph nodes connected by relations such as `HAS_TURN` and `HAS_MESSAGE`. A ContextBuilder can traverse from a session node to assemble context. The measured graph traversal latency (BFS 1.20 ms and DSL Chain Depth30 79.3 μs) supports the feasibility of this design, but not a complete agent-system claim.

For graph-heavy workloads, the current configuration changes NodeStore/EdgeStore reads from pagecache + ArcSwap to direct mmap pointer reads. It further maintains a per-RelationType

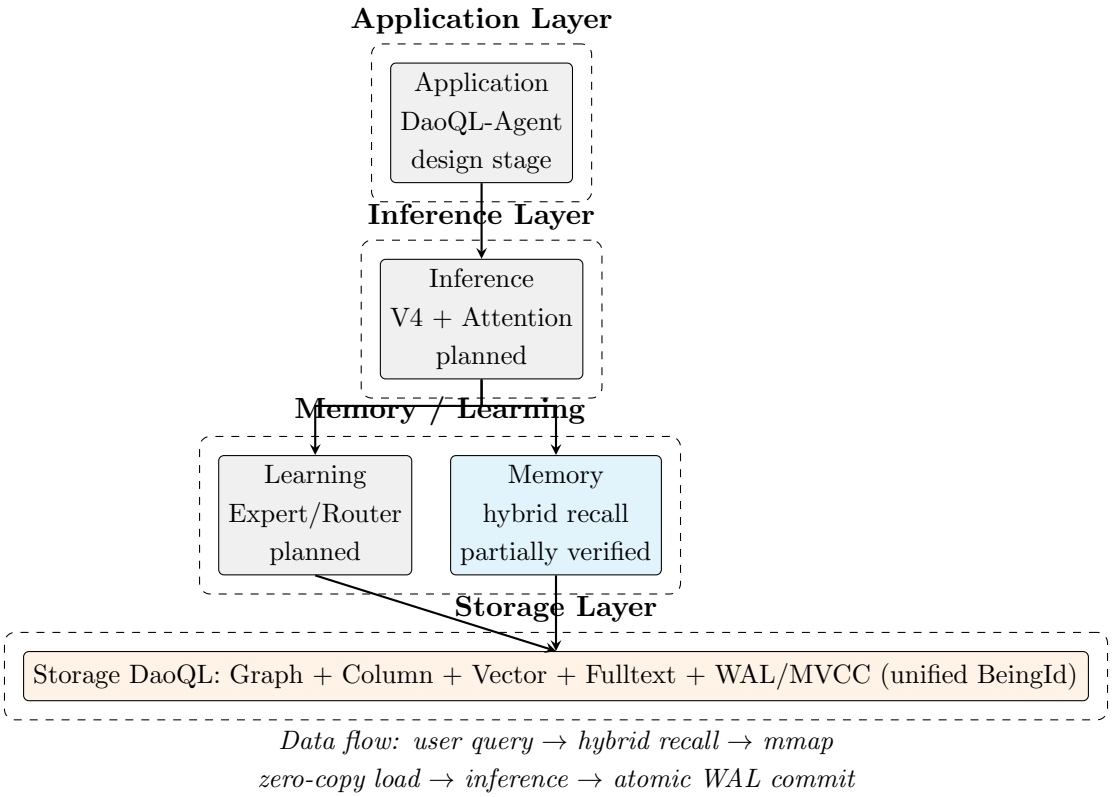


Figure 1: Five-layer neuro-symbolic architecture and verification scope. The storage layer is measured in the current configuration; the memory layer is indirectly supported by component benchmarks; application, inference, and learning layers are future targets.

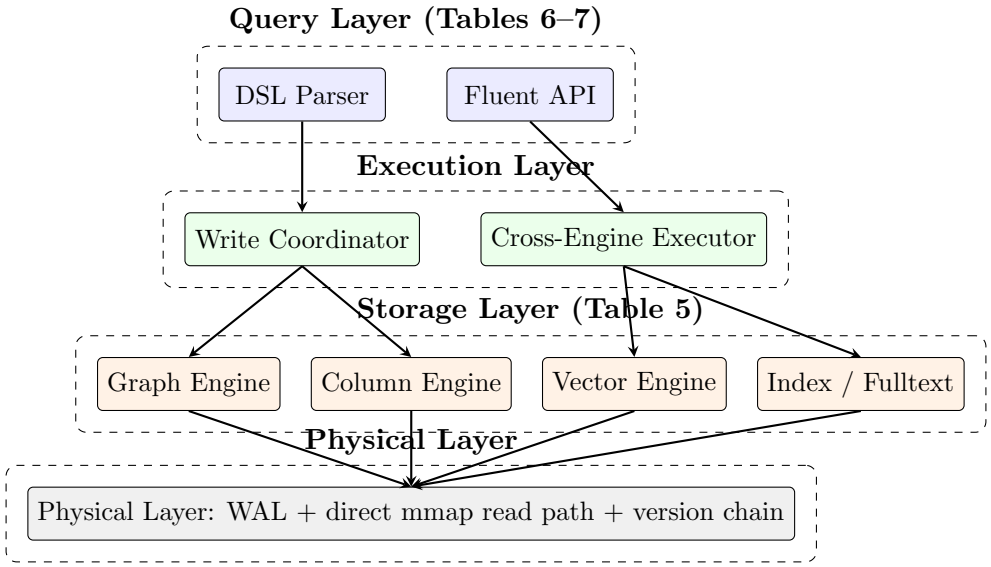


Figure 2: DaoQL's internal four-layer engine architecture and its mapping to the three-tier benchmark suite. The current measurement introduces direct mmap reads in the physical layer and P0#1/#2/#5 write-path optimizations.

CSR adjacency file (`csr_{:04x}.bin`) so type-filtered BFS and triangle-count queries can read neighbors without scanning edge chains. This reduces edge write-read latency from 397.6 ns to 304.4 ns and BFS fanout4 depth5 from 1.97 ms to 1.20 ms. In LDBC SNB, IC14 drops from the 30 s range to 0.64 ms.

3.3 Experimental Design

3.3.1 Environment and Datasets

Microbenchmarks and same-machine competitor comparisons run on an Apple M-series machine with 64 GB LPDDR5 and NVMe SSD (∼3.5 GB/s), Rust 1.94.1, Criterion.rs release mode, and median reporting. LDBC SNB and ANN-Benchmarks exploratory runs use Apple M4 Max, 128 GB LPDDR5, and NVMe SSD (∼7 GB/s), DaoQL v1.1.0 release build. The two groups are reported separately and are not merged into a single ranking.

- **XY-ERP**: synthetic ERP data, about 1.9M entities covering orders, BOM, and supplier links.
- **YAGO4**: public knowledge graph, about 3.4M entities, used for reproducibility checks.
- **Synthetic microbenchmarks**: 10K vectors (64D, Cosine, $M = 16$, ef=100) and 10K Chinese documents.
- **LDBC SNB SF1**: about 1M Persons, 37M edges, and 34 IC/BI queries, using a custom driver rather than official submission.
- **ANN-Benchmarks**: `random-s-100-euclidean`, 90K × 100 training vectors, 10K × 100 test vectors.

Table 3: Fairness boundaries of performance comparisons

Dimension	Setting	Interpretation boundary
Deployment shape	DaoQL embedded; competitors often client-server	Ratios include process, protocol, and serialization costs
Data scale	10K–100K microbenchmarks; 1.9M business entities	Not evidence for billion-scale workloads
Cache state	Criterion release median; same-machine repeated runs	Hot-cache and tail latency need separation
HNSW parameters	10K vectors, 64D, $M = 16$, ef=100	Only this parameter range
Hybrid query	Single-process vector → graph → column	No full multi-stack RAG baseline
Standard sets	M4 Max 128GB vs M-series 64GB	Reported separately, not merged
LDBC status	34/34 queries run	Not official LDBC certification

3.3.2 Benchmark and Counterfactual Protocols

We use three benchmark tiers: sub-crate engine microbenchmarks, Fluent API microbenchmarks, and DSL end-to-end microbenchmarks. Business stress tests include engine, Fluent, and DSL suites. Configuration ablation compares the previous pagecache/ArcSwap configuration with the current mmap + P0 write-path configuration. LDBC SNB uses SF1, 8 workers, 30 s warmup, 600 s measurement, and 1 GB page cache. ANN-Benchmarks reports QPS and Recall@10 for ef_search $\in \{50, 100, 200, 400, 800, 1600\}$.

The counterfactual experiment compares GPT-4o with DaoQL+GPT-4o. It covers five domains - medical decision, financial credit, supply-chain risk, regulatory compliance, and dynamic pricing - with 50 cases per domain and five natural-language paraphrases per case, for 1,250 queries. Metrics include answer consistency, safety, reasoning-chain traceability, update correctness after new information, counterfactual stability, and composable counterfactual decomposability. The materials describe domains, scale, metrics, and partial ground-truth rules, but not full prompts, temperature, scoring protocol, or per-case error logs; therefore we treat the result as preliminary evidence.

4 Results and Analysis

We evaluate four aspects: counterfactual robustness, storage performance, same-machine and standard-set benchmarks, and system-level primitive composition. Values in Sections 4.2–4.3 use the M-series 64 GB platform; Section 4.4 uses the M4 Max 128 GB platform. All DaoQL measurements use real engine calls and no mock or stub paths.

4.1 Counterfactual Consistency

This experiment tests the prediction of Section 3.1.5: if natural language maps to a patch and executes $\mathsf{Eval}(\phi, \delta(S, p))$, composed changes should satisfy Eq. 1; GPT-4o alone lacks δ and structured state.

Table 4: Counterfactual consistency experiment across five domains ($n = 1250$)

Metric	**GPT-4o**	**DaoQL+GPT-4o**	**Gain**
Answer consistency	71%	97%	+26%
Safety	79%	95%	+16%
Traceable reasoning chain	—	100%	—
Correct update after new information	61%	100%	+39%
Counterfactual stability	58%	96%	+38%
Composable counterfactual decomposability	45%	94%	+49%

The gap between 94% and 45% on composable decomposability is consistent with Theorem 1 and Observation 3. The remaining 6% failures in the explicit path are attributed in the source materials to parsing errors, under-encoded rules, or DSL boundaries; without per-case error logs, we do not further decompose the failure modes quantitatively.

4.2 Three-Tier Microbenchmarks and Ablation

Tables 5–7 compare the previous configuration with the current mmap read path and P0 write-path optimizations.

Table 5: Engine-level ablation: previous vs. current configuration

Benchmark	Previous	Current	Change
graph / node_store_alloc_1000	10.81 µs	7.17 µs	+34%
graph / node_store_write_read	1.86 µs	1.24 µs	+33%
graph / edge_store_write_read	397.6 ns	304.4 ns	+23%
column / raw_append_1000	9.03 ms	4.16 ms	+54%
column / projected_scan_10000	377.2 µs	221.7 µs	+41%
write / append_single	16.70 µs	4.74 µs	+72%
write / append_batch_10000	50.0 ms	14.91 ms	+70%
vector / hnsw_search_k10	95.4 µs	61.3 µs	+36%
vector / persistence_roundtrip	11.13 ms	1.66 ms	+85%
lookup (B+Tree)	253.3 ns	160.7 ns	+37%

Table 6: Fluent API microbenchmark ablation

Benchmark	Previous	Current	Change
graph_write_and_traverse	18.26 µs	16.57 µs	+9%
column_write_and_scan	389.4 µs	374.4 µs	+4%
vector_search	110.6 µs	98.4 µs	+11%
fulltext_search	482.0 µs	415.6 µs	+14%
index_uuid_lookup	178.1 ns	186.9 ns	−5%

Table 7: DSL microbenchmark ablation

Benchmark	Previous	Current	Change
graph_traversal / 1k_depth3	25.27 µs	21.40 µs	+15%
combined / material_order	46.16 µs	36.54 µs	+21%
column_aggregate / 5k_groupby	1.20 ms	1.11 ms	+8%
graph_node_lookup / 1k_full	3.52 µs	3.17 µs	+10%

The largest gains come from removing per-row flushes and bypassing pagecache/ArcSwap on graph reads. Improvements attenuate at the Fluent and DSL layers, indicating that wrapper overhead and Being assembly remain optimization targets.

4.3 Competitor Comparison and Fairness Boundaries

Table 8 summarizes same-machine results. Because DaoQL is embedded while most competitors are client-server systems, these are engineering end-to-end comparisons rather than pure kernel comparisons.

The full-text result is an explicit weakness: Tantivy+jieba at 1.27 ms is much slower than PostgreSQL's simple tsvector configuration at 32 µs, largely because jieba performs Chinese

Table 8: DaoQL current configuration vs. best observed competitor

Capability	DaoQL	Best comparator	Observation
Point lookup	0.38 μs	RocksDB $\sim$1 μs	Embedded low latency
Full BFS traversal	1.20 ms	Neo4j 5–8 ms	Lower end-to-end latency
HNSW k=10	83.1 μs	Qdrant 363.4 μs	Leads under current parameters
Column SUM 100K	362.9 μs	ClickHouse $\sim$1 ms	Leads at current scale
Batch write	1.75 μs/row	PostgreSQL 9.0 μs/row	Embedded low latency
Chinese full text 10K	1.27 ms	PostgreSQL tsvector 32 μs	DaoQL is $\sim$40$\times$ slower
Cross-engine hybrid query	105.8 μs	No direct comparator	Missing multi-stack baseline

Table 9: HNSW head-to-head: DaoQL vs. Qdrant 1.16, same machine and parameters

System	HNSW k=10	Brute Force k=10	SIMD
DaoQL current	83.1 μs	269.0 μs	8$\times$f32
DaoQL previous	148.4 μs	388.3 μs	8$\times$f32
Qdrant 1.16	363.4 μs	283.4 μs	16$\times$f32

segmentation while the simple baseline does not. This should be treated as a limitation rather than hidden in aggregate performance claims.

4.4 Standard-Set Exploratory Measurements

LDBC SNB SF1. DaoQL runs all 34 IC/BI queries on SF1 with zero errors under an 8-worker, 600 s run. RelationType CSR adjacency improves type-filtered traversal and triangle-count workloads. Representative results appear in Table 10. IC-style entity lookup and bounded traversal are generally interactive, but IC6/IC10, BI4, and BI9 remain severe long-tail bottlenecks. Overall throughput is only 1.8 QPS, so SNB BI workloads are not an advantage claim.

Table 10: Representative LDBC SNB SF1 latencies (DaoQL v1.1.0, M4 Max)

Class	Query	P50 (ms)	P95 (ms)	Interpretation
IC	IC1 profile	0.26	7.00	Interactive lookup
IC	IC2/IC8 recent messages/comments	0.16	0.36	Interactive
IC	IC13 shortest path	0.05	2.88	Multi-hop traversal
IC	IC3 recommendation	1.74	10.13	Acceptable
IC	IC6/IC10 thread/recommendation	$\sim$18,900	$\sim$26,000	Long-tail bottleneck
BI	BI8 social circle	7.98	59.28	Moderate
BI	BI12/BI17 triangles	4,813/5,222	8,288/8,708	Improved but slow
BI	BI4 hot forums	21,225	31,483	Long-tail bottleneck
BI	BI9 forum tags	908,271	969,790	Extreme long tail
Overall	34/34 coverage, 0 errors	1.8 QPS		Dragged by slow queries

ANN-Benchmarks. On `random-s-100-euclidean`, DaoQL HNSW with $M = 32$, $M_0 = 64$, and ef_construction=200 reports the curve in Table 11. At ef_search=200, Recall@10 is 99.3% at 5,724 QPS. The v1.1.0 bridge-edge protection fix raises Recall@200 from about 32% to above 99%, so correctness is the main result. Public hnswlib/FAISS numbers come from

different hardware and are not used as same-machine wins.

Table 11: ANN-Benchmarks `random-s-100-euclidean` (DaoQL v1.1.0)

ef_search	QPS	Recall@10	Zero-recall queries
50	19,001	94.4%	511/10000
100	12,225	97.4%	218/10000
200	5,724	99.3%	42/10000
400	3,051	99.9%	2/10000
800	1,529	100.0%	0/10000
1600	708	100.0%	0/10000

4.5 Business Stress Tests and Hybrid Queries

Table 12: `xy_erp_stress` engine-level results

Scenario	Scale	Current	Change
BFS Fanout4 Depth5	1,365 nodes	1.20 ms	+39%
Order Status Chain	30 hops	2.03 ms	+27%
Hybrid vector → graph → column	10K	119.09 µs	+14%
Write Batch With-Vector	16t×10K	1,009.2 ms	+14%
WAL Recovery	10K	219.6 µs	+20%

Table 13: Fluent API stress tests: BOM traversal and batch write

Scenario	Previous	Current	Gain
Chain Depth5 Skip Ext	19.0 µs	14.1 µs	+26%
Chain Depth30	144.5 µs	129.5 µs	+10%
Tree Fanout4 Depth5	6.45 ms	5.45 ms	+16%
Raw Engine BFS Fanout4 Depth5	1.82 ms	1.23 ms	+32%
Write Batch 1K No Embed	46.9 ms	23.7 ms	+49%
Hybrid Query	108.1 µs	105.8 µs	+2%

The hybrid query demonstrates the promise of a shared BeingId in one process, but because we do not provide a Neo4j+Qdrant+ClickHouse multi-stack RAG baseline, it is reported as an architectural potential rather than a completed cross-system victory.

4.6 Composite Business Scenarios

DaoQL-Edu integration tests validate Being, Def, Relation, and Version composition in five representative scenarios.

4.7 Optimization Stability

The pagecache/ArcSwap design improved some batch paths but introduced traversal and single-write regressions. The current mmap + P0 write configuration restores read, write, and vector paths simultaneously.

Table 14: DSL stress tests: BOM traversal and mutation

Scenario	**Previous**	**Current**	**Gain**
Chain Depth30	111.7 μs	79.3 μs	+29%
Tree Fanout4 Depth5	3.59 ms	2.99 ms	+17%
Write Batch 1K No-Vec	81.5 ms	25.1 ms	+69%
Mutation No-Vec 1K	22.16 ms	8.58 ms	+61%

Table 15: Unit-test-driven composite scenarios

Scenario	**Question**	**Engine path**	**Result**
Sales report	Order total / filtered SUM	Column fast path + SIMD	1M rows, 3.4 μs
Risk detection	Similar trade + linked account	Vector $\rightarrow$ graph $\rightarrow$ column	non-empty result
Recommendation	Similar user + influencer ranking	Vector + graph + column	priority=80.0
Version replay	Entity price history	Version chain + MVCC	old version preserved
Batch consistency	100 Beings + chain relations	WAL + BFS + aggregate	node count matches

Table 16: Configuration-level ablation and stability

Configuration	**Baseline**	**Feature**
Early unified read path	+O-1–O-4	Establishes unified index and read baseline
warm cache / BQ	+warm_cache, BQ	Point lookup improves but graph traversal regresses
pagecache + ArcSwap	+ArcSwap, pagecache batch	Batch improves but single write/HNSW/Fluent write regress
mmap + P0 write optimization	+mmap, P0#1/#2/#5	Current configuration with positive metrics

5 Discussion

5.1 Theory-Design-Experiment Closure

Table 17: Theory, design, and experiment closure

Theory	**DaoQL design**	**Prediction**	**Status**
Prop. 1	`DAO_DEF` axiom layer	Schema cannot recurse forever	Theoretical closure
Prop. 2	Fuel + sandbox	Write path does not hang	Theoretical closure
Obs. 1	BeingId explicitness	LLM has no local update guarantee	Counterfactual comparison
Prop. 3	WAL + MVCC	Single update does not spread	Version-chain tests
Thm. 1	Deterministic DSL Eval	High composable consistency	94%
Obs. 3	GPT-4o baseline	Prompt-only path is unstable	45%
Eq. 2	Single-process engines	Low hybrid latency	105.8 μs
BFS complexity	mmap pointer adjacency	Constant-factor advantage	1.20 ms

This table separates proofs from measurements. For example, $O(|V'|+|E'|)$ BFS complexity does not imply a 4.2× Neo4j speedup; the latter is an engineering constant-factor measurement on one setup.

5.2 Architectural Verification Status

Table 18: Architectural decisions and current verification status

Decision	**Measurement evidence**	**Status**
Join-free graph engine	BFS 1.20 ms, 4.2–6.7× Neo4j	Verified
Direct mmap read path	edge 304.4 ns, BFS +39%	Verified
Relation-type CSR adjacency	LDBC IC14 0.64 ms, BI12/17 improved	Verified
ProjectedLayer + SIMD column	SUM 362.9 μs, 2.8× ClickHouse	Verified
BeingId zero-copy across engines	Hybrid 105.8 μs	Verified
WAL crash consistency	Recovery 10K 219.6 μs	Verified
KVCache graph nodes	Architecture plan	Design stage
Expert hot update	Architecture plan	Design stage
DaoQL-Agent context traversal	Design + BFS support	Design stage

5.3 Positioning Against Industrial Ontology Platforms

Industrial platforms such as Palantir Foundry demonstrate that explicit Ontology plus AI has commercial value in large enterprises and government settings. Their strengths are federated data integration, low-code Agent Studio, and a mature ecosystem. DaoQL should be positioned as complementary, not as a direct replacement: it targets lighter deployment, data-sovereign environments, and write-path contracts in medium-scale or edge settings. We do not compare customer counts, TCO, or production maturity. The architectural distinction is narrower: an overlay Ontology eases legacy integration, while embedded Being/Type/Relation makes Eval and WAL audit semantics easier to close in one transaction model.

5.4 Limitations

1. **Embedded vs. client-server.** DaoQL avoids network overhead that competitors often include. Ratios should not be read as kernel-only superiority.

2. **Scale.** Most microbenchmarks are 10K–100K; billion-scale graphs or vector indexes may change relative performance.

3. **Chinese full text.** Tantivy+jieba is a clear weakness compared with PostgreSQL simple tsvector, although the tokenization work is not equivalent.

4. **Concurrent writes.** 16-connection throughput drops about 40%; sharded locks are future work.

5. **Counterfactual experiment.** The protocol lacks full prompts, temperature, complete scoring rules, and per-case logs; results are preliminary.

6. **LDBC SNB.** This is a custom-driver coverage experiment, not official certification; 1.8 QPS and BI9/IC6 long tails show immaturity on global social-network aggregation workloads.

7. **ANN-Benchmarks.** Only one synthetic dataset is used, public competitor QPS is cross-hardware, and pre-fix v1.0 vector numbers should not be reused.

8. **Industrial-platform comparison.** Palantir ecosystem maturity is not experimentally evaluated; discussion is architectural positioning only.

5.5 Future Work

Priorities include optimizing LDBC SNB long-tail BI/IC query plans and caches, migrating to wider SIMD paths for vectors, adding sharded locks and Contract caching for concurrent writes, improving block-level column vectorization, implementing Fluent Being assembly zero-copy, expanding counterfactual experiments to 2,500 cases with complete scoring protocols, and validating DaoQL's deeper integration with V4 inference.

6 Conclusion

This paper addresses four structural crises caused by encoding world models implicitly in LLM weights. It proposes data-first ontology: deterministic knowledge is represented by explicit primitives in DaoQL, while the LLM serves as a language and reasoning auxiliary. The formal framework defines explicit world models, proves finite Def bootstrapping and strong Contract termination, and gives a sufficient condition for composable counterfactual decomposability. The system implementation validates the storage layer through graph, column, vector, full-text, WAL/MVCC, direct mmap reads, and cross-engine hybrid queries.

On the engineering side, the current Apple M-series measurements show graph BFS at 1.20 ms, HNSW at 83.1 μs, and Fluent hybrid query at 105.8 μs. M4 Max exploratory standard-set runs add LDBC SNB 34/34 query coverage and ANN-Benchmarks Recall@10 $\geq$ 99% at thousand-level QPS, while also exposing SNB BI long tails and cross-hardware limits. On reasoning robustness, the five-domain counterfactual experiment reports 94% composable counterfactual decomposability for DaoQL+GPT-4o, 49 percentage points above GPT-4o alone. The paper therefore presents a theory-experiment path toward auditable, modifiable, knowledge-accumulating world models, while explicitly marking agent runtime, KVCache graph nodes, and expert hot updates as future work.

References

[1] AlphaProof Team. AI achieves silver-medal standard solving international mathematical olympiad problems. Google DeepMind Blog, 2024.

[2] Martin Aumüller, Erik Bernhardsson, and Alexander Faithfull. ANN-Benchmarks: A benchmarking tool for approximate nearest neighbor algorithms. *Information Systems*, 87:101374, 2020.

[3] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. In *International Conference on Learning Representations*, 2015.

[4] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.

[5] Artur d'Avila Garcez and Luís C. Lamb. Neurosymbolic AI: The 3rd wave. *Artificial Intelligence Review*, 56:12387–12406, 2023.

[6] Nicola Guarino. Formal ontology, conceptual analysis and knowledge representation. *International Journal of Human-Computer Studies*, 43(5-6):625–640, 1995.

[7] Zirui Guo, Lianghao Xia, Yanhua Yu, Tu Ao, and Chao Huang. LightRAG: Simple and fast retrieval-augmented generation. *arXiv preprint arXiv:2410.05779*, 2024.

[8] David Ha and Jürgen Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2018.

[9] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022.

[10] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

[11] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Yejin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38, 2023.

[12] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

[13] Yann LeCun. A path towards autonomous machine intelligence. OpenReview position paper, 2022.

[14] Hector J. Levesque and Ronald J. Brachman. A fundamental tradeoff in knowledge representation and reasoning. In *Readings in Knowledge Representation*, pages 41–70. Morgan Kaufmann, 1985.

[15] Linked Data Benchmark Council. LDBC social network benchmark (SNB). Benchmark specification, 2015.

[16] John McCarthy. Programs with common sense. In *Proceedings of the Teddington Conference on the Mechanization of Thought Processes*, pages 77–84. 1959.

[17] Michael McCloskey and Neal J. Cohen. Catastrophic interference in connectionist networks: The sequential learning problem. In *Psychology of Learning and Motivation*, volume 24, pages 109–165. Academic Press, 1989.

[18] Marvin Minsky. A framework for representing knowledge. In *The Psychology of Computer Vision*, pages 211–277. McGraw-Hill, 1975.

[19] Neo4j Inc. Neo4j graph database. Release 2025.04.0, 2025. Accessed 2026-05-31.

[20] Allen Newell and Herbert A. Simon. GPS, a program that simulates human thought. In *Computers and Thought*, pages 279–293. McGraw-Hill, 1963.

[21] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

[22] Palantir Technologies. Palantir foundry ontology. Product documentation, 2024.

[23] Judea Pearl. *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. Morgan Kaufmann, 1988.

[24] M. Ross Quillian. Semantic memory. In *Semantic Information Processing*, pages 227–270. MIT Press, 1968.

[25] Tim Rocktäschel and Sebastian Riedel. End-to-end differentiable proving. In *Advances in Neural Information Processing Systems*, 2017.

[26] Werner Schuster. Rich hickey's Datomic embraces cloud, intelligent applications and consistency. InfoQ News, 2012. Accessed 2026-05-31.

[27] Luciano Serafini and Artur d'Avila Garcez. Logic tensor networks: Deep learning and logical reasoning from data and knowledge. *arXiv preprint arXiv:1606.04422*, 2016.

[28] Vaticle Ltd. TypeDB: A polymorphic database for knowledge systems, 2024. Accessed 2026-05-31.

[29] Jim Webber. *Graph Databases: New Opportunities for Connected Data*. O'Reilly Media, 2013.

[30] World Wide Web Consortium. OWL 2 web ontology language document overview. Technical report, W3C Recommendation, 2012.

[31] World Wide Web Consortium. SPARQL 1.1 query language. Technical report, W3C Recommendation, 2013.

[32] World Wide Web Consortium. RDF 1.1 concepts and abstract syntax. Technical report, W3C Recommendation, 2014.